Системная аналитика и архитектура декларативной платформы оркестрации ИИ-агентов

Внедрение больших языковых моделей (LLM) в корпоративные среды привело к фундаментальному сдвигу в парадигме автоматизации. Традиционные платформы интеграции, такие как n8n или Zapier, опираются на жестко детерминированные графы выполнения и визуальные конструкторы, где каждый шаг алгоритма программируется человеком. Однако по мере роста сложности задач эти монолитные процессы становятся хрупкими, а перенос всей логики ветвления на единую языковую модель приводит к переполнению контекстного окна, деградации рассуждений и так называемому «гниению контекста» (context rot). Для решения кросс-функциональных задач требуется переход к модульным мультиагентным системам (Multi-Agent Systems, MAS), где специализированные автономные узлы координируются через стандартизированные протоколы межсервисного взаимодействия.
Проектирование масштабируемой системы оркестрации, лишенной недостатков жестких визуальных конструкторов на этапе ядра, требует разделения архитектуры на уровень управления (Control Plane) и уровень исполнения (Data Plane). Основная концепция заключается в полной инкапсуляции поведения агента, его инструментов и внутреннего графа переходов в единый декларативный JSON-контракт. В сочетании с динамическим обнаружением агентов через семантический поиск, автоматическим разрешением зависимостей и поддержкой современных стандартов, таких как Model Context Protocol (MCP), подобная система способна универсально обогащать входящий контент и служить надежным фундаментом для будущего внедрения No-Code интерфейсов.

Концептуальная архитектура платформы

Анализируемая система строится на взаимодействии трех фундаментальных компонентов: реестра агентов, сервиса макро-оркестрации и сервиса изолированного микро-исполнения. Архитектурное разделение позволяет масштабировать каждый компонент независимо и гарантирует, что когнитивная нагрузка на языковые модели распределяется равномерно.

Компонент	Архитектурная роль	Основная функция в системе	Технологический паттерн
База агентов	Реестр знаний	Хранение JSON-схем, метаданных и векторов компетенций агентов. Обеспечение семантического поиска.	Векторная база данных (OpenSearch), Hybrid Search
Оркестратор	Control Plane	Семантический анализ входящего контента,	Macro-Orchestration, Topological Sorting, Agent-to-Agent (A2A)

Компонент	Архитектурная роль	Основная функция в системе	Технологический паттерн
		обнаружение релевантных агентов, разрешение графа зависимостей (DAG), координация потока данных между агентами.	
Проигрыватель	Execution Plane	Получение JSON-манифеста конкретного агента, развертывание его внутреннего графа, выполнение LLM-запросов, вызов внешних инструментов (REST, MCP, Kafka), валидация ответа.	Micro-Execution, State Machine, Model Context Protocol Client

Данный паттерн, известный как «Оркестратор-Воркер» (Orchestrator-worker pattern), централизует делегирование задач, позволяя при этом изолированным агентам-воркерам (в нашем случае запускаемым внутри Проигрывателя) специализироваться на конкретных доменах без знания о глобальном состоянии системы.

Реестр агентов и алгоритмы семантического обнаружения

В динамической мультиагентной среде система не может полагаться на статические идентификаторы для маршрутизации задач. Входящий контент зачастую представляет собой неструктурированный текст (например, электронное письмо клиента, лог системной ошибки или бизнес-запрос), который необходимо сопоставить с конкретным исполнителем. Для решения этой задачи применяется база данных OpenSearch, выступающая в роли интеллектуального реестра агентов.
Использование OpenSearch с функционалом k-NN (k-nearest neighbors) векторов позволяет реализовать семантический поиск (Semantic Search). В отличие от классического лексического поиска, семантический поиск оценивает контекст и намерение (intent), сопоставляя запрос пользователя с векторным представлением описания компетенций агента. Если поступает запрос на «сверку квартальных балансов», система сможет найти агента, описанного как «Специалист по финансовому аудиту и бухгалтерии», даже если точные слова не совпадают.
Для обеспечения максимальной точности извлечения агентов индекс OpenSearch должен быть спроектирован с учетом гибридного подхода, комбинирующего плотные векторы (dense vectors) для семантики и инвертированные индексы (BM25) для точного лексического совпадения.

Поле индекса OpenSearch	Тип данных	Описание и назначение
agent_id	keyword	Уникальный детерминированный идентификатор агента (например, finance_auditor_v1).
name	text	Человекочитаемое имя агента для визуализации и лексического поиска.
description	text	Подробное описание компетенций, бизнес-логики и ограничений агента.
capabilities_embedding	knn_vector	Плотный вектор (например, 1536 измерений для моделей OpenAI или 768 для локальных трансформеров), сгенерированный на основе поля description.
de[span_15](start_span)[span_15](end_span)pendencies	keyword (array)	Массив идентификаторов других агентов, от вывода которых зависит данный узел.
manifest	object	Полная декларативная JSON-структура агента, передаваемая в сервис Проигрывателя.

Процесс обнаружения инициируется Оркестратором через специализированный внутренний инструмент (AgentDiscoveryTool). При получении входящего контента Оркестратор векторизует текст запроса с использованием настроенной модели эмбеддингов, формирует запрос к OpenSearch и получает отсортированный по релевантности список кандидатов. Выбранный кандидат предоставляет свой массив зависимостей и манифест для дальнейшей обработки.

Декларативная JSON-структура агента

Основой переносимости, безопасности и предсказуемости системы является JSON-схема агента. Опираясь на стандарты Open Agent Specification и спецификации декларативных агентов (например, Microsoft Declarative Agents), архитектура системы предполагает концепцию «конфигурация вместо кода» (configuration-over-code). Исключение императивного программирования из определения агента позволяет в будущем легко интегрировать визуальный конструктор, который будет выступать исключительно в роли графического интерфейса для генерации и редактирования этого JSON-документа. Структура агента представляет собой строгий контракт, описывающий интерфейсы ввода-вывода, системные инструкции, доступные инструменты и внутреннюю машину состояний (граф переходов).

Описание входных и выходных данных (I/O Contracts)

Для обеспечения надежного взаимодействия между Оркестратором и независимыми агентами, каждый манифест должен строго типизировать ожидаемый контекст и гарантировать формат возвращаемых данных. Это реализуется через интеграцию стандарта JSON Schema (RFC 8259) непосредственно в определение агента.
Секции input_schema и output_schema определяют контракты данных. Оркестратор перед вызовом Проигрывателя проверяет, соответствуют ли накопленные данные требуемой input_schema целевого агента. В свою очередь, Проигрыватель, завершая выполнение внутреннего графа агента, использует методы принудительного структурированного вывода (например, response_format в современных LLM API или генерацию с ограничениями грамматики), чтобы заставить языковую модель вернуть результат, строго соответствующий output_schema. Это полностью устраняет проблему нестабильного парсинга свободного текста и гарантирует детерминированность конвейера.

Системные промпты и ролевые модели

В структуре агента описывается секция prompts, содержащая базовый контекст для языковой модели. Здесь фиксируются системные инструкции (System Instructions), описывающие роль агента, ограничения безопасности (Guardrails), формат рассуждений (например, требование использовать Chain-of-Thought) и стратегии обработки ошибок. Вынесение этих параметров в декларативный JSON позволяет версионировать личность и поведение агента независимо от базового кода Проигрывателя.

Внутренний граф переходов (Micro-Graph)

Монолитные промпты, в которых агенту предоставляется доступ ко всем инструментам одновременно с инструкцией «реши задачу», демонстрируют низкую надежность в корпоративных средах. Декларативная структура решает эту проблему путем внедрения внутреннего микро-графа (Micro-Graph), который определяет изолированные шаги (узлы) и условные переходы (ребра).

Элемент микро-графа	Описание	Логика выполнения в Проигрывателе
Nodes (Узлы)	Изолированные этапы выполнения внутри агента. Узел может иметь тип reasoning (запрос к LLM для генерации плана), tool_execution (выполнение конкретного API-вызова) или data_transformation (преобразование форматов).	Проигрыватель выполняет логику узла, обновляя локальное состояние агента (State). На каждом узле LLM получает доступ только к тем инструментам, которые разрешены в контексте данного шага.
Edges (Ребра)	Направленные связи между узлами, определяющие последовательность выполнения.	Проигрыватель перемещается от узла к узлу на основе статических связей или вычисляемых условий.
Conditions (Условия)	Логические выражения на ребрах, позволяющие реализовать динамическую	Если узел фильтрации спама возвращает is_spam: true, условное ребро может

Элемент микро-графа	Описание	Логика выполнения в Проигрывателе
	маршрутизацию (Skip Logic).	направить граф сразу к завершающему узлу, минуя ресурсоемкие узлы глубокого анализа, что экономит вычислительные ресурсы.

Наличие внутреннего графа позволяет агенту выполнять сложные циклические операции, такие как паттерн рефлексии (Reflection), где агент генерирует результат, отправляет его на узел оценки (критики), и в случае неудовлетворительного качества возвращается на этап генерации с обновленным контекстом ошибок.

Интеграция и декларативное описание инструментов (Tools)

Способность агента взаимодействовать с внешним миром определяется набором его инструментов. Ключевая проблема существующих платформ заключается в необходимости написания императивного связующего кода (glue code) для каждой новой интеграции. Предлагаемая архитектура решает эту проблему, позволяя полностью декларировать логику вызова инструментов внутри JSON-манифеста агента. Проигрыватель берет на себя задачу интерпретации этих деклараций и выполнения реальных сетевых или локальных вызовов.
Система поддерживает четыре фундаментальных класса инструментов: встроенные утилиты, декларативные REST API интеграции, Model Context Protocol (MCP) серверы и интерфейсы потоковой передачи данных Kafka.

Встроенные инструменты (Built-in Tools)

Для обеспечения базовых вычислительных потребностей в платформу (на уровне Проигрывателя) интегрирован набор безопасных, локально выполняемых утилит. К ним относятся калькуляторы математических выражений, модули форматирования дат, инструменты криптографического хеширования и парсеры регулярных выражений. Эти инструменты не требуют описания сетевой логики в JSON агента; достаточно указать идентификатор инструмента (например, builtin:calculator). Проигрыватель перехватывает запрос от LLM к такому инструменту и выполняет вычисление в изолированной песочнице, возвращая результат в контекст.

Декларативная интеграция REST API

Значительная часть корпоративной инфраструктуры базируется на RESTful архитектуре, однако оборачивание каждого эндпоинта в отдельный MCP-сервер или написание кода на Python/JavaScript нецелесообразно. Система позволяет вписывать логику выполнения REST API запроса непосредственно в JSON агента, используя спецификации, вдохновленные стандартами OpenAPI и JSON Schema.
В манифесте инструмента типа rest_api декларируются метод HTTP (GET, POST, PATCH), базовый URL, конфигурация заголовков и правила аутентификации. Критически важным

элементом является свойство parameters_schema, которое использует стандарт JSON Schema для строгого определения того, какие аргументы LLM должна сгенерировать для этого вызова.
Когда внутренний граф агента доходит до узла выполнения REST API, процесс выглядит следующим образом:

1. Проигрыватель предоставляет LLM описание инструмента и его parameters_schema.
2. LLM генерирует JSON-объект с необходимыми параметрами (например, идентификатор клиента и период дат).
3. Проигрыватель валидирует полученный JSON против схемы (чтобы предотвратить инъекции и ошибки типизации).
4. Выполняется интерполяция шаблонов: аргументы из JSON подставляются в URL (query parameters) или тело запроса (payload).
5. Проигрыватель извлекает конфиденциальные ключи из защищенного хранилища (Vault) и добавляет их в заголовки запроса.
6. Выполняется сетевой запрос. Ответ сервера (часто избыточный) фильтруется с использованием выражений JSONPath (описанных в манифесте инструмента), чтобы вернуть в контекст LLM только релевантные поля, избегая переполнения токенов.

Этот механизм делает архитектуру полностью независимой от разработчиков: аналитик может добавить новый инструмент, просто описав его REST-контракт в JSON агента.

Интеграция Model Context Protocol (MCP)

В сценариях, где REST API слишком сложен (например, требует глубокой пагинации, сохранения состояния соединения или взаимодействия с локальными файловыми системами), система использует Model Context Protocol (MCP). MCP — это открытый стандарт клиент-серверного взаимодействия, который позволяет абстрагировать реализацию инструментов в отдельные микросервисы.
Вместо того чтобы описывать логику каждого запроса, в JSON агента указывается лишь подключение к MCP-серверу (например, по протоколу HTTP+SSE или стандартному вводу-выводу). Сервис Проигрывателя включает в себя полнофункциональный MCP-клиент. При запуске агента Проигрыватель отправляет запрос tools/list к указанному MCP-серверу. Сервер возвращает стандартизированный список доступных инструментов, их описаний и схем ввода. Проигрыватель динамически пробрасывает этот манифест в контекст LLM. Когда LLM принимает решение использовать инструмент, Проигрыватель отправляет команду tools/call на MCP-сервер, который выполняет всю тяжелую работу (например, выполнение SQL-запроса к базе данных) и возвращает результат.
Этот подход радикально снижает сложность JSON-манифеста агента и позволяет переиспользовать один MCP-сервер (например, сервер взаимодействия с GitHub) множеством разных агентов.

Асинхронные потоки и инструменты Apache Kafka

Для поддержки высоконагруженных и событийно-ориентированных (event-driven) архитектур в платформу внедрена декларативная поддержка Apache Kafka. Традиционные REST API синхронны, что может вызывать проблемы с таймаутами при длительных рассуждениях агентов. Интеграция Kafka позволяет агентам асинхронно «забирать» (consume) или «класть» (produce) данные в брокер сообщений.

В JSON-схеме агента Kafka-инструмент описывается указанием топика, роли (producer/consumer) и схемы сериализации.

- **Режим Producer (Положить данные):** Если по бизнес-логике агент должен инициировать фоновый процесс (например, отправить уведомление на генерацию PDF), он формирует JSON-сообщение согласно определенной в инструменте payload_schema. Проигрыватель сериализует это сообщение (часто с валидацией через Confluent Schema Registry, чтобы гарантировать совместимость версий контракта данных) и отправляет его в кластер Kafka.
- **Режим Consumer/Trigger (Забрать данные):** Подключение к Kafka также может служить триггером для запуска мультиагентной цепочки на уровне Оркестратора. Появление нового события (сообщения) в топике маршрутизируется Оркестратором на соответствующего агента.

Использование Kafka-инструментов позволяет выстраивать отложенные приоритетные очереди, разделять медленные и быстрые потоки обработки (throttling) и сохранять полную трассировку (audit trail) действий агентов в распределенных системах.

Макро-оркестрация и алгоритмы разрешения графа зависимостей

Самой сложной задачей в мультиагентных системах является координация работы независимых узлов. Когда сложные задачи декомпозируются, возникает ситуация, при которой целевой агент не может выполнить свою работу без входных данных, которые должен сгенерировать другой агент. Согласно требованиям, структура агента может зависеть от выполнения другого, и при обнаружении такого агента система должна «дойти по цепочке» до независимого агента и проиграть всю последовательность. Эту задачу решает сервис Оркестрации, применяя алгоритмы теории графов.

Математическая модель макро-графа (Macro-DAG)

Зависимости между агентами формируют направленный ациклический граф (Directed Acyclic Graph, DAG) на макроуровне. В этом графе узлами выступают сами агенты, а направленные ребра обозначают потоки данных (Data Flow) и зависимости выполнения. Если пользователь запрашивает комплексный анализ, Оркестратор с помощью семантического поиска в OpenSearch находит конечного агента A (Аналитик). Однако в JSON-манифесте агента A в массиве dependencies указан агент B (Агрегатор метрик), а у агента B — агент C (Извлекатель сырых данных).

Алгоритм поиска корневого узла и планирования (Root Search & Topological Sort)

Для разрешения этой цепочки Оркестратор реализует алгоритм обратной трассировки (Backward Tracing) в комбинации с топологической сортировкой.

1. **Инициализация и обратный обход (Depth-First Search):** Начиная с целевого агента A, Оркестратор анализирует его массив dependencies. Он рекурсивно или итеративно обращается к базе OpenSearch для извлечения манифестов каждого зависимого агента. Процесс (поиск в глубину) продолжается до тех пор, пока

алгоритм не обнаружит узлы, у которых массив dependencies пуст.

2. **Обнаружение корневых агентов:** Агенты без зависимостей (в нашем примере агент C) идентифицируются как корневые узлы (Root Agents). Это точки входа в вычислительный процесс, поскольку им требуются только исходные данные пользователя, а не артефакты других агентов.
3. **Проверка на цикличность (Cycle Detection):** Критически важным этапом является проверка графа на наличие циклических зависимостей (когда агент A зависит от B, а B зависит от A). Если алгоритм обхода встречает узел, который уже находится в текущем пути поиска, Оркестратор прерывает планирование с ошибкой Deadlock Detected, так как циклы в макро-DAG приводят к бесконечному ожиданию (starvation).
4. **Топологическая сортировка (Kahn's Algorithm):** После построения полного дерева зависимостей, Оркестратор применяет алгоритм Кана (или аналогичный алгоритм топологической сортировки). В результате многомерный граф преобразуется в упорядоченную очередь или волны выполнения (Execution Waves). Все корневые узлы помещаются в первую волну, узлы, зависящие только от корневых — во вторую, и так далее.
5. **Построение плана выполнения (Execution Plan):** Результатом работы алгоритма является план: [C],, [A]. Узлы внутри одного подмассива (волны) не имеют взаимных зависимостей и могут быть отправлены в Проигрыватель параллельно (Parallel Orchestration Pattern). Узлы в последующих массивах ожидают завершения предыдущих (Sequential Orchestration).

Диспетчеризация и маршрутизация контекста

После формирования плана Оркестратор переходит в фазу активного дирижирования (Planner-Executor Pattern). Он передает JSON-манифест корневого агента C и пользовательский контекст в сервис Проигрывателя. После успешного завершения Проигрыватель возвращает структурированный JSON-ответ. Оркестратор сохраняет этот ответ в промежуточное хранилище состояний (State Store).
Затем Оркестратор вызывает агента B, передавая ему не только оригинальный запрос, но и точные выходы агента C, сопоставляя их с input_schema агента B. Этот процесс продолжается до тех пор, пока целевой агент A не завершит работу, после чего итоговый результат возвращается пользователю. Данный подход гарантирует надежность, так как каждый агент оперирует только в зоне своей компетенции, а Оркестратор контролирует общую сборку системы.

Сервис изолированного исполнения (Проигрыватель)

Сервис Проигрывателя (Player / Execution Engine) — это вычислительное ядро системы. В отличие от Оркестратора, который видит картину глобально, Проигрыватель работает в строгой изоляции. Его единственная задача — получить JSON-манифест одного конкретного агента, инициализировать его контекст, выполнить внутренний микро-граф (Micro-Graph) и гарантированно вернуть результат, валидный согласно выходной схеме.

Управление состоянием и транзакциями (State Machine)

Проигрыватель функционирует как конечный автомат (State Machine). Получив манифест, он парсит секции nodes и edges. Каждый переход между узлами сопровождается обновлением локального состояния агента (Local State). Если узел представляет собой LLM-вызов, Проигрыватель компилирует промпты (включая системные инструкции и историю диалога), внедряет доступные на этом шаге инструменты (преобразуя их JSON Schema в формат, понятный OpenAI/Anthropic API) и отправляет запрос провайдеру. При получении ответа от LLM с намерением вызвать инструмент (Tool Call), Проигрыватель приостанавливает выполнение генерации, парсит параметры вызова и направляет их в соответствующий обработчик (модуль REST API, MCP-клиент или драйвер Kafka). Полученный от инструмента результат (Tool Message) добавляется в контекст, и управление возвращается LLM для дальнейшего анализа. Этот цикл (Reason-Act) продолжается до достижения терминального узла графа.

Защита от сбоев (Resilience Patterns)

Взаимодействие с внешними API и LLM сопряжено с высокой вероятностью сетевых ошибок, таймаутов или галлюцинаций моделей. Для обеспечения надежности (Enterprise-grade reliability) Проигрыватель реализует следующие механизмы:

- **Circuit Breakers и Retries:** При сбоях вызова REST-инструментов применяется экспоненциальная задержка (Exponential Backoff), чтобы не перегружать сторонние серверы.
- **Ограничение итераций (Loop Bounds):** Для предотвращения бесконечных циклов вызова инструментов (когда агент постоянно получает ошибку и пытается вызвать инструмент снова) устанавливается жесткий лимит итераций на каждый узел. При его превышении граф переходит на узел обработки ошибок (Fallback Node).
- **Авто-коррекция схем (Self-Correction):** Если LLM генерирует выходные данные, которые не проходят валидацию Pydantic или JSON Schema (например, отсутствует обязательное поле), Проигрыватель перехватывает исключение и отправляет его обратно в LLM с системным промптом, указывающим на конкретную ошибку валидации, заставляя модель исправить свой ответ (Reflection Pattern). Это обеспечивает железобетонную гарантию того, что Оркестратор получит корректно структурированный ответ.

API Контракты межсервисного взаимодействия

Для обеспечения слабой связности (loose coupling) и возможности масштабирования, взаимодействие между компонентами системы — базой агентов (OpenSearch), Оркестратором и Проигрывателем — осуществляется через строгие REST или gRPC интерфейсы.

Контракт 1: Инициация оркестрации (Client → Orchestrator)

Внешняя система (например, вебхук или чат-интерфейс) отправляет неструктурированный запрос в Оркестратор для инициации процесса. **POST /api/v1/orchestrator/dispatch** *Запрос (Payload):*

```
{
  "request_id": "req-9876-uuid",
  "content": "Собери данные по инциденту #404 и подготовь RCA отчет,
отправь его в Kafka.",
  "metadata": {
    "tenant_id": "enterprise_a",
    "priority": "high"
  }
}
```

Ответ (Response):

```
{
  "status": "accepted",
  "orchestration_job_id": "job-1234-uuid",
  "links": {
    "status": "/api/v1/orchestrator/jobs/job-1234-uuid/status"
  }
}
```

Контракт 2: Семантическое обнаружение (Orchestrator → OpenSearch)

Оркестратор переводит текст в эмбеддинг и ищет релевантных кандидатов в базе агентов. **POST /agents_index/_search** *Запрос (Payload):*

```
{
  "size": 5,
  "_source": ["agent_id", "dependencies", "manifest"],
  "query": {
    "knn": {
      "capabilities_embedding": {
        "vector": [0.015, -0.022, 0.104],
        "k": 5
      }
    }
  }
}
```

Контракт 3: Микро-исполнение агента (Orchestrator → Player)

Оркестратор передает изолированную задачу Проигрывателю после разрешения графа зависимостей. **POST /api/v1/player/execute** *Запрос (Payload):*

```
{
  "execution_id": "exec-5555-uuid",
  "agent_manifest": {...Полная JSON-структура агента... },
  "input_context": {
```

```
    "incident_id": "#404",
    "raw_logs": "[2026-03-25 18:50:00] ERROR Database connection
failed"
  },
  "execution_limits": {
    "max_tokens": 8000,
    "timeout_ms": 60000
  }
}
```

Ответ (Response): Возвращает строго валидированный результат в формате, заданном в output_schema агента.

```
{
  "status": "success",
  "output_data": {
    "rca_report": "Причина инцидента #404 кроется в обрыве соединения
с БД...",
    "severity_score": 8.5
  },
  "telemetry": {
    "tokens_consumed": 2450,
    "tools_invoked": ["search_logs_mcp", "kafka_producer"]
  }
}
```

Анализ технологического стека и вариантов реализации

Разработка столь сложной распределенной системы с нуля несет значительные риски. Существуют проверенные open-source фреймворки для построения рабочих процессов (Workflow Engines) и координации агентов, которые могут лечь в основу архитектуры. Выбор фундаментального технологического стека определяет надежность, простоту развертывания и возможности масштабирования платформы.

Технология / Фреймворк	Роль в системе	Преимущества	Недостатки / Риски
Temporal.io	Макро-Оркестратор	Долгоживущее выполнение (Durable Execution). Автоматическое сохранение состояния между вызовами агентов, встроенные механизмы повторных попыток (retries) и тайм-аутов. Идеально	Сложность инфраструктуры (требует развертывания кластера серверов Temporal и базы данных). Высокий порог входа.

Технология / Фреймворк	Роль в системе	Преимущества	Недостатки / Риски
		для разрешения сложного макро-DAG.	
Apache Airflow	Макро-Оркестратор	Мощный планировщик пакетных (batch) DAG-ов. Огромное комьюнити и готовые операторы.	Ориентирован на запуск задач по расписанию (cron-style), плохо подходит для динамического построения графов в реальном времени (event-driven), что критично для агентов.
LangGraph	Движок Проигрывателя	Нативная интеграция с экосистемой LangChain, поддержка графовых структур (циклы) в памяти, встроенные функции потоковой передачи данных и управления контекстом LLM.	При использовании в качестве глобального оркестратора синхронен и теряет устойчивость при масштабных сбоях (OOM). Идеален только на уровне микро-исполнения.
Apache Kafka	Шина данных (Event Bus)	Полностью децентрализованная архитектура. Позволяет компонентам общаться через топики (Producer/Consumer), обеспечивая балансировку нагрузки и отказоустойчивость.	Повышенная задержка (latency) по сравнению с REST. Сложность отладки (требуется распределенная трассировка).
FastAPI / Python	Базовый стек API	Высокая производительность за счет asyncio, строгая валидация контрактов через Pydantic (идеально для проверки JSON Schema). Стандарт де-факто в ИИ.	Однопоточность Python может потребовать горизонтального масштабирования (Kubernetes) для обработки тысяч агентов одновременно.

Рекомендуемая архитектурная гипотеза

Для достижения баланса между надежностью энтерпрайз-уровня и гибкостью разработки

ИИ-систем, наиболее оптимальным вариантом является комбинированный стек: **Temporal + LangGraph + FastAPI**.
В этой парадигме **Temporal** выполняет роль Оркестратора. Получив запрос, Temporal инициирует рабочий процесс (Workflow), который опрашивает OpenSearch, строит макро-DAG зависимостей и диспетчеризует задачи (Activities). Каждое Activity — это вызов сервиса Проигрывателя (написанного на **FastAPI**). Внутри Проигрывателя разворачивается локальный граф на базе **LangGraph**, который динамически формируется из переданного JSON-манифеста агента. LangGraph берет на себя циклы взаимодействия с LLM и инструментами (MCP, REST), а FastAPI гарантирует валидацию схем ввода/вывода через Pydantic. Если микро-граф Проигрывателя падает из-за сетевой ошибки API или таймаута, Temporal автоматически восстанавливает задачу на другом узле без потери общего прогресса глобальной цепочки.
Kafka интегрируется не как основная шина связи сервисов, а как набор специфических инструментов агентов, позволяя им взаимодействовать с внешними корпоративными системами по событийно-ориентированной модели.

Стратегия интеграции визуального конструктора

Глобальная цель описанной архитектуры — создание надежного вычислительного ядра, к которому в дальнейшем будет подключен визуальный No-Code конструктор агентов, делающий платформу доступной для бизнес-пользователей.
Благодаря строгой декларативной природе ядра, внедрение визуального интерфейса становится тривиальной задачей. В отличие от систем, где визуальные узлы компилируются в непрозрачный код (например, старые версии интеграционных шин), здесь пользовательский интерфейс (написанный, например, на базе React Flow или Vue Flow) выступает исключительно в роли визуализатора и редактора JSON-манифеста.

- **Граф агента:** Узлы и ребра визуального канваса напрямую транслируются (two-way binding) в массивы nodes и edges внутри секции graph JSON-документа.
- **Инструменты:** Добавление инструмента — это генерация блока в массиве tools. Для REST API платформа может предоставлять формы, которые автоматически генерируются на основе JSON Schema (JSON Forms), скрывая сложность конфигурации заголовков и маппинга переменных.
- **MCP-интеграции:** При подключении MCP-сервера визуальный интерфейс может автоматически отобразить все доступные на сервере функции, опрашивая endpoint tools/list, превращая сложную настройку в процесс вида «Plug-and-Play».

Такой подход гарантирует, что система останется архитектурно чистой: вычислительные микросервисы (Оркестратор и Проигрыватель) никогда не зависят от особенностей реализации фронтенда, а сам фронтенд оперирует исключительно стандартизированными, версионируемыми JSON-схемами. В результате платформа получает масштабируемость корпоративных workflow-движков, гибкость передовых мультиагентных фреймворков и простоту визуальной сборки на уровне лучших No-Code альтернатив.

Источники

1. Multi-Agent Systems: Orchestrating AI Agents with A2A Protocol, https://medium.com/@yusufbaykaloglu/multi-agent-systems-orchestrating-ai-agents-with-a2a-pr

otocol-19a27077aed8 2. Multi-agent Systems Explained in 17 Minutes, https://www.youtube.com/watch?v=Mi5wOpAgixw 3. AI Agent Orchestration Patterns - Azure Architecture Center | Microsoft Learn, https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns 4. The Orchestration of Multi-Agent Systems: Architectures, Protocols, and Enterprise Adoption, https://arxiv.org/html/2601.13671v1 5. Simpliflow: A Lightweight Open-Source Framework for Rapid Creation and Deployment of Generative Agentic AI Workflows - arXiv, https://arxiv.org/html/2510.10675v2 6. woodyhayday/FlowSpec: FlowSpec: Automation Workflow Schema - A lightweight JSON schema for defining automations and multi-step workflows. Designed for AI Automation Workflows - GitHub, https://github.com/woodyhayday/FlowSpec 7. I Tried 20+ MCP (Model Context Protocol) Courses on Udemy: Here are My Top 5 Recommendations for…, https://medium.com/javarevisited/i-tried-20-mcp-model-context-protocol-courses-on-udemy-here-are-my-top-5-recommendations-for-921440120326 8. The 10 Best n8n Enterprise Alternatives for Workflow Automation - Adopt AI, https://www.adopt.ai/blog/n8n-alternatives 9. Four Design Patterns for Event-Driven, Multi-Agent Systems - Confluent, https://www.confluent.io/blog/event-driven-multi-agent-systems/ 10. OpenSearch semantic search: The basics and a quick tutorial [2026 guide], https://www.instaclustr.com/education/opensearch/opensearch-semantic-search-the-basics-and-a-quick-tutorial-2026-guide/ 11. Agentic search - OpenSearch Documentation, https://docs.opensearch.org/latest/vector-search/ai-search/agentic-search/index/ 12. Semantic search - OpenSearch Documentation, https://docs.opensearch.org/latest/vector-search/ai-search/semantic-search/ 13. Build a LangGraph Multi-Agent system in 20 Minutes with LaunchDarkly AI Configs, https://launchdarkly.com/docs/tutorials/agents-langgraph 14. Configuring agents for semantic search - OpenSearch Documentation, https://docs.opensearch.org/latest/vector-search/ai-search/agentic-search/neural-search/ 15. Open Agent Specification (Agent Spec) Technical Report - arXiv, https://arxiv.org/html/2510.04173v3 16. The Auton Agentic AI Framework A Declarative Architecture for Specification, Governance, and Runtime Execution of Autonomous Agent Systems - arXiv, https://arxiv.org/html/2602.23720v1 17. Introducing JSON Forms in Appsmith, https://www.appsmith.com/blog/introducing-json-forms-in-appsmith 18. Declarative agent schema 1.5 for Microsoft 365 Copilot, https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/declarative-agent-manifest-1.5 19. Exploring a contract-driven alternative to agent loops (reducers + orchestrators + declarative execution) : r/LangChain - Reddit, https://www.reddit.com/r/LangChain/comments/1phlfoh/exploring_a_contractdriven_alternative_to_agent/ 20. From Chaos to Structure: Building Production-Ready AI Agents with Guaranteed JSON Responses | by Velu Sankaran | Medium, https://medium.com/@v31u/from-chaos-to-structure-building-production-ready-ai-agents-with-guaranteed-json-responses-dfd925bad7ea 21. JSON Schema - REST API Tutorial, https://restfulapi.net/json-schema/ 22. API Endpoint Calling Tool Guidelines for Generative AI Agents - Oracle Help Center, https://docs.oracle.com/en-us/iaas/Content/generative-ai-agents/api-calling-tool-guidelines.htm 23. Talk Freely, Execute Strictly: Schema-Gated Agentic AI for Flexible and Reproducible Scientific Workflows - arXiv, https://arxiv.org/html/2603.06394v1 24. Introducing Structured Outputs for Batch and Agent Workflows | Databricks Blog, https://www.databricks.com/blog/introducing-structured-outputs-batch-and-agent-workflows 25.

From Words to Structure: Generating JSON with GenAI | by Akanksha Sinha - Medium, https://medium.com/@akankshasinha247/from-words-to-structure-generating-json-with-genai-0494b7d46eda 26. Agentic AI vs Deterministic Workflows with LLM Components : r/ExperiencedDevs - Reddit, https://www.reddit.com/r/ExperiencedDevs/comments/1nqlm09/agentic_ai_vs_deterministic_workflows_with_llm/ 27. Declarative agent schema 1.0 for Microsoft 365 Copilot, https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/declarative-agent-manifest-1.0 28. awesome-copilot/instructions/declarative-agents-microsoft365.instructions.md at main - GitHub, https://github.com/github/awesome-copilot/blob/main/instructions/declarative-agents-microsoft365.instructions.md 29. AI Agent Anti-Patterns (Part 1): Architectural Pitfalls That Break Enterprise Agents, https://achan2013.medium.com/ai-agent-anti-patterns-part-1-architectural-pitfalls-that-break-enterprise-agents-before-they-32d211dded43 30. How to build a multi-agent system using Elasticsearch and LangGraph, https://www.elastic.co/search-labs/blog/multi-agent-system-llm-agents-elasticsearch-langgraph 31. MCP Explained: The Protocol Connecting AI Agents to Everything, https://thecraftman.medium.com/mcp-explained-the-protocol-connecting-ai-agents-to-everything-f7c2f745b9d9 32. APIs in the Age of AI Agents - Medium, https://medium.com/@viajesubmarino/apis-in-the-age-of-ai-agents-1b4215e138a8 33. Patterns for web APIs and REST APIs - Azure Logic Apps | Microsoft Learn, https://learn.microsoft.com/en-us/azure/logic-apps/logic-apps-create-api-app 34. Add skills to a declarative agent created with Microsoft 365 Agents Toolkit - Microsoft Learn, https://learn.microsoft.com/en-us/microsoft-365/copilot/extensibility/build-declarative-agents-add-skills 35. Extend your agent with tools from a REST API (preview) - Microsoft Copilot Studio, https://learn.microsoft.com/en-us/microsoft-copilot-studio/agent-extend-action-rest-api 36. Exploring an Idea for an APIs.json Defined Workflow - API Evangelist, https://apievangelist.com/2023/11/01/apis-json-workflows/ 37. Making REST APIs Agent-Ready: From OpenAPI to Model Context Protocol Servers for Tool-Augmented LLMs - arXiv.org, https://arxiv.org/html/2507.16044v1 38. How to Implement Tool Schemas - OneUptime, https://oneuptime.com/blog/post/2026-01-30-tool-schemas/view 39. Making AI Useful: How to Use json_schema and Function (via. tools) in the Responses API, https://medium.com/@arda.arslan/making-ai-useful-how-to-use-json-schema-and-function-via-tools-in-the-responses-api-a36568ab6694 40. Tool Calling Explained: The Core of AI Agents (2026 Guide) - Composio, https://composio.dev/content/ai-agent-tool-calling-guide 41. Level up your REST API's with JSON Schema - This Dot Labs, https://www.thisdot.co/blog/level-up-your-rest-apis-with-json-schema 42. The Many Amazing Uses of JSON Schema: Client-side Validation - Medium, https://medium.com/apis-you-wont-hate/the-many-amazing-uses-of-json-schema-client-side-validation-c78a11fbde45 43. REST API - Appsmith docs, https://docs.appsmith.com/connect-data/reference/rest-api 44. Tool calling authentication for AI agents - Scalekit, https://www.scalekit.com/blog/tool-calling-authentication-ai-agents 45. What Is the Model Context Protocol (MCP) and How It Works - Descope, https://www.descope.com/learn/post/mcp 46. What is Model Context Protocol (MCP)? A guide | Google Cloud, https://cloud.google.com/discover/what-is-model-context-protocol 47. Extend your agent with Model Context Protocol - Microsoft Copilot Studio, https://learn.microsoft.com/en-us/microsoft-copilot-studio/agent-extend-action-mcp 48. Build an MCP client - Model Context Protocol, https://modelcontextprotocol.io/docs/develop/build-client

49. Code execution with MCP: building more efficient AI agents - Anthropic, https://www.anthropic.com/engineering/code-execution-with-mcp 50. How Kafka improves agentic AI | Red Hat Developer, https://developers.redhat.com/articles/2025/06/16/how-kafka-improves-agentic-ai 51. Building AI API Interfaces: From REST to ML-Optimized Design - Gravitee, https://www.gravitee.io/blog/ai-api-interface-design-rest-to-ml 52. Introduction | Apache Kafka, https://kafka.apache.org/documentation/ 53. Building an event-driven agentic AI system with Apache Kafka on Confluent Cloud and watsonx Orchestrate - IBM Developer, https://developer.ibm.com/tutorials/event-driven-agentic-ai-system-confluent-watsonx-orchestrate/ 54. Kafka connections - AWS Glue, https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-etl-connect-kafka-home.html 55. Kafka Producer and Consumer - Medium, https://medium.com/@cobch7/kafka-producer-and-consumer-f1f6390994fc 56. Kafka Schema Registry and JSON Schema: A Comprehensive Guide | Pravin on Software, https://pravin.dev/posts/kafka-schema-registry-and-json-schema-a-comprehensive-guide/ 57. How to Use JSON Schema with Kafka - OneUptime, https://oneuptime.com/blog/post/2026-01-21-kafka-json-schema/view 58. Producing and consuming JSON in KAFKA - Stack Overflow, https://stackoverflow.com/questions/39691176/producing-and-consuming-json-in-kafka 59. What is AI Agent Orchestration? - IBM, https://www.ibm.com/think/topics/ai-agent-orchestration 60. How do multi-agent systems manage task dependencies? - Milvus, https://milvus.io/ai-quick-reference/how-do-multiagent-systems-manage-task-dependencies 61. VillagerAgent: A Graph-Based Multi-Agent Framework for Coordinating Complex Task Dependencies in Minecraft - arXiv.org, https://arxiv.org/html/2406.05720v1 62. Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems - arXiv, https://arxiv.org/html/2603.14688v1 63. Directed Acyclic Graphs: The Backbone of Modern Multi-Agent AI | by Dr. Santanu Bhattacharya, https://santanub.medium.com/directed-acyclic-graphs-the-backbone-of-modern-multi-agent-ai-d9a0fe842780 64. GraphTracer: Graph-Guided Failure Tracing in LLM Agents for Robust Multi-Turn Deep Search - arXiv, https://arxiv.org/html/2510.10581v1 65. Network Graphs for Dependency Resolution | Towards Data Science, https://towardsdatascience.com/network-graphs-for-dependency-resolution-5327cffe650f/ 66. How to resolve a DAG of module dependencies? - Stack Overflow, https://stackoverflow.com/questions/53154360/how-to-resolve-a-dag-of-module-dependencies 67. MAPRO: Recasting Multi-Agent Prompt Optimization as Maximum a Posteriori Inference, https://arxiv.org/html/2510.07475v1 68. TALM: Dynamic Tree-Structured Multi-Agent Framework with Long-Term Memory for Scalable Code Generation - arXiv, https://arxiv.org/html/2510.23010v1 69. dag-dependency-resolver | Skills Mar... - LobeHub, https://lobehub.com/skills/curiositech-windags-skills-dag-dependency-resolver 70. Multi-Agent System Reliability: Failure Patterns, Root Causes, and Production Validation Strategies - Maxim AI, https://www.getmaxim.ai/articles/multi-agent-system-reliability-failure-patterns-root-causes-and-production-validation-strategies/ 71. Efficient algorithm for finding multi-source children in a DAG? - Stack Overflow, https://stackoverflow.com/questions/73364691/efficient-algorithm-for-finding-multi-source-children-in-a-dag 72. Orchestrating Multi-Agent Systems: How Do Agents Decide Who Goes First? | by Sathya Pramod Bhat Muniyooru Nataraja | Medium, https://medium.com/@sathya.nataraja/orchestrating-multi-agent-systems-how-do-agents-decide

-who-goes-first-61b91871bf2d 73. Orchestration Patterns & Multi-Agent Workflows | AB-100 Exam Prep (Ep 4.2), https://www.youtube.com/watch?v=TPSqS6ltifE 74. Multi-Agent System Patterns: A Unified Guide to Designing Agentic Architectures - Medium, https://medium.com/@mjgmario/multi-agent-system-patterns-a-unified-guide-to-designing-agentic-architectures-04bb31ab9c41 75. Temporal + LangGraph: A Two-Layer Architecture for Multi-Agent Coordination - Anup Jadhav, https://www.anup.io/temporal-langgraph-a-two-layer-architecture-for-multi-agent-coordination/ 76. 10 Multi-Agent Coordination Strategies to Prevent System Failures - Galileo AI, https://galileo.ai/blog/multi-agent-coordination-strategies 77. A DAG-Based Approach to LLM Workflow Orchestration - DEV ..., https://dev.to/ivan_holovach_f2abf13a514/a-dag-based-approach-to-llm-workflow-orchestration-1i98 78. LangGraph — Structuring LLM Tool Calls with Pydantic and JSON Serialization - Medium, https://medium.com/@shuv.sdr/langgraph-structuring-llm-tool-calls-with-pydantic-and-json-serialization-1715f7a0c2e0 79. Top Open Source Workflow Orchestration Tools in 2025 - Bytebase, https://www.bytebase.com/blog/top-open-source-workflow-orchestration-tools/ 80. A curated list of awesome open source workflow engines - GitHub, https://github.com/meirwah/awesome-workflow-engines 81. Need advice in Structuring JSON Output in Langgraph for Chatbot : r/LangChain - Reddit, https://www.reddit.com/r/LangChain/comments/1dna58k/need_advice_in_structuring_json_output_in/ 82. I built an n8n alternative focused specifically on AI Agents & Visual Workflows. I need you to roast it (Alpha). : r/SaaS - Reddit, https://www.reddit.com/r/SaaS/comments/1pelxuk/i_built_an_n8n_alternative_focused_specifically/ 83. JSON Form - Appsmith docs, https://docs.appsmith.com/reference/widgets/json-form 84. Agentic AI Frameworks: Architectures, Protocols, and Design Challenges - arXiv, https://arxiv.org/html/2508.10146v1 85. 15 Best n8n Alternatives in 2026 - Vellum AI, https://vellum.ai/blog/best-n8n-alternatives